

半仓智拼 (HalfCart) —— 七大业务流程实现原理详解

本文档只讲“怎么实现的”（原理/机制/步骤），不涉及具体代码。目的是让你能把每个流程从头到尾讲清楚——数据从哪来、经过哪些步骤、最后落到哪。

流程一：语音发单 —— 语音 → 文字 → 结构化订单

整体链路（四个阶段）

用户说话 → 【采集】 → 【转写成文字】 → 【AI结构化抽取】 → 【生成订单草稿】

阶段①：采集（浏览器端做的事）

用户点击麦克风按钮的瞬间，浏览器同时启动**两条采集通道**，谁先出结果就用谁：

- 通道A（保底）**：浏览器调用 MediaRecorder 接口，把麦克风的聲音持续录制成一段音频数据（比如WAV/OGG格式），存在浏览器内存里，用户点“停止”就打包成一个音频文件。这条通道**在任何浏览器/任何环境下都能用**，是兜底方案。
- 通道B（加速器，仅部分浏览器支持）**：浏览器同时调用 Web Speech API，这是浏览器自带的实时语音识别能力（背后调用的是浏览器厂商自己的语音识别服务，比如Chrome用Google的）。它会**边说边实时出文字结果**，速度很快（不到1秒），而且完全免费，不需要经过你自己的服务器。

为什么两条一起跑：微信小程序内嵌浏览器不支持通道B，所以必须有通道A兜底；但如果用户环境支持通道B，它比通道A快得多（通道A还要把音频传到服务器转写，多一轮网络传输和计算时间），所以两条一起跑，谁先给结果就直接用谁的，另一条的结果被丢弃。

阶段②：转写成文字（只有走通道A才需要这一步）

如果最终用的是通道A（录音文件），浏览器把这段音频上传到后端服务器，后端要把“一段声音”转换成“一串文字”，这一步叫“语音识别/ASR”。转写路径设计成两级：

- 优先尝试云端语音识别服务（阿里云NLS）**：把音频发给阿里云的语音识别接口，云端模型帮你转写，准确率更高（约95%），但这条路径目前还没有申请真实密钥，处于“代码写好但未接入验证”的状态。
- 实际生效的是本地小模型（Whisper tiny）**：这是一个开源的、体积很小（约几十MB）的语音识别模型，直接跑在你自己的服务器上（不需要联网调第三方），CPU就能推理，几秒内出结果。准确率比云端方案低一些（约85%），但胜在不依赖任何外部服务、不花钱、不用等密钥审批。

3. 转写出来的文字会做一个简单检查：统计里面中文字符占比，如果占比过低（说明可能是噪音被识别成了乱码或者识别错成了外语），直接判定为无效识别，拒绝往下走，让用户重新录制。

阶段③：AI结构化抽取（这是"AI理解意图"的核心步骤）

不管文字是从通道B实时来的，还是通道A转写出来的，最终都会变成一段自然语言文字，比如："山姆瑞士卷一盒，3个人拼，每人35，明天下午3点南门见"。

这段文字本身对系统来说毫无结构，系统需要知道的是："商品名叫什么、需要几个人、每人多少钱、什么时候截止、在哪交接"这五个具体字段。这一步交给大语言模型（DeepSeek）来做：

1. 把这段文字连同一份"指令说明"一起发给DeepSeek的对话接口。这份指令说明里写清楚了：你要从用户的话里提取哪几个字段、每个字段是什么含义、返回格式必须是什么样的JSON结构、遇到"明天""下午3点"这种模糊说法要怎么换算成具体的日期时间。
2. DeepSeek 理解这段自然语言后，按指令要求的格式，把这句话"翻译"成一份结构化的JSON数据，比如 {"商品名": "山姆瑞士卷", "人数": 3, "人均价": 35, ...}。
3. 系统拿到这份JSON后，用一套预先定义好的数据结构（Pydantic模型）去校验它——每个字段的类型对不对、必填字段是不是都有值。如果校验通过，说明这份AI输出是"结构合规"的，可以往下用。

这一步实际会遇到的问题：大模型有时候不会乖乖只返回一份纯JSON，它可能在JSON前后加一些解释性的话，或者用markdown的代码块把JSON包起来。系统这里做了"三层兜底"：先直接按纯JSON去解析，失败了就试着从markdown代码块里把JSON部分抠出来解析，再失败就用文本匹配的方式，从整段文字里硬找出被大括号包起来的那一块内容再解析。三层里任何一层成功了就用那个结果。

阶段④：置信度判断 + 生成订单草稿

AI在返回结构化数据的同时，还会给一个"置信度"分数（0到1之间，代表AI自己对这次提取结果有多大把握）。系统在这个基础上再做一层**自己的**规则校验，而不是完全相信AI给的置信度：

- 如果AI提取出的人数小于2，系统会自动强制改成2（因为拼单至少要2人），但这个"被迫修正"的动作本身说明这次提取不太可靠。
 - 如果价格是0或负数、或者没提取出交接地點，系统会主动把置信度往下压低。
 - 最终只有置信度达到一个阈值（0.6），并且几个关键字段都不为空，才会判定"可以自动填表"——把这份AI生成的结构化数据直接填进发布订单的表单里，用户看到的是一份已经填好的表单，只需要检查确认再点提交，而不是自己从头一个字段一个字段填。
 - 如果置信度不够，系统仍然会尽量填一部分能确定的字段，但会明确提示用户"某些信息不确定，请检查/补充"，把最终决定权交还给用户，而不是让一个不确定的AI结果直接变成正式订单。
-

流程二：雷达大厅 —— 附近拼单怎么被找到、怎么排序

整体链路

用户打开首页 → 【获取用户当前坐标】 → 【按坐标+半径查询附近订单】 → 【按距离+信用分排序】 → 【结果缓存后返回】

阶段①：获取用户坐标

前端调用高德地图的JS SDK，拿到用户当前所在的经纬度坐标（这一步依赖浏览器的定位权限，用户需要授权）。

阶段②：地理范围查询（核心机制）

系统里每一个拼单订单，在创建的时候都会存一个"交接地点对应的经纬度坐标"。查询附近订单，本质上是一个"以用户当前坐标为圆心，画一个半径N公里的圆，看看哪些订单的坐标落在这个圆里"的地理计算问题。

这类计算如果直接用普通数据库（MySQL）做，效率很低——因为普通数据库没有专门为"地理位置范围搜索"设计索引结构，做一次这种查询相当于要把很多条记录都拿出来逐条算距离，数据量大了会很慢。

所以这个查询放在 **Elasticsearch**（一个专门做搜索和检索的引擎）里做。做法是：把每个订单的坐标以一种叫 `geo_point`（地理点）的专用数据类型存进Elasticsearch，Elasticsearch原生支持"给定一个中心点和半径，直接筛出落在这个范围内的所有记录"这种查询方式（叫 `geo_distance` 查询），而且底层有专门的地理索引结构支撑，查询效率比普通数据库高很多。

阶段③：排序（距离优先，信用分次要）

筛出"附近的订单"之后，还要决定"谁排在前面"。规则是**先按离用户的距离从近到远排，距离相同或接近的情况下，再按发起这个拼单的团长的信用分从高到低排**。这个排序逻辑也是Elasticsearch原生支持的，可以同时指定两个排序字段和优先级。

阶段④：省钱和防抖动的两个附加设计

- **减少调用第三方地图接口的次数**：如果要精确计算两点之间的实际距离，理论上可以调高德地图的距离计算接口，但那是要花钱、有调用频率限制的第三方服务。系统改成本地用一个数学公式（Haversine球面距离公式）直接根据两点的经纬度算出直线距离，够用且不花钱不限速。
- **给查询结果加缓存**：同一个区域的用户短时间内反复刷新首页，如果每次都重新查一次Elasticsearch，会给数据库层造成压力。所以查询结果会先存进Redis缓存一小段时间（10秒），10秒内同一个区域的请求直接用缓存结果，不重新查。这里有个小技巧：缓存的"key"（缓存的标识）不是用户精确到小数点后好几位的坐标，而是把坐标"抹去"到大约100米的精度再作为key——这样即使两个用户站的位置差几十米，也能命中同一份缓存，缓存利用率更高。

- 另外，地图这类第三方服务偶尔会抽风（超时、报错），系统给这类调用加了一个“熔断”保护：如果连续多次调用都失败，系统会判定“这个服务现在不太行”，接下来一段时间内直接不再尝试调用它（避免浪费时间等一个大概率会失败的请求），而是直接用模拟数据顶上，等冷却时间过了再重新尝试。

流程三：加入拼单 —— 怎么防止抢名额抢出问题

场景问题

假设一个拼单还差最后1个名额，A和B两个人几乎同时点了“加入”。如果系统处理不当，可能出现：两个人都被判定“加入成功”，导致这个拼单变成超员（比如本来要5人，结果收了6份钱）；或者扣款成功了但名额数没更新对，数据变得不一致。

实现原理：把“加入”变成一个不可被打断的整体操作

核心思路是：在同一时刻，只允许一个“加入”请求真正执行完整的三个步骤（查名额→扣款→改人数），其他同时到达的请求必须排队等待，等前一个执行完才能继续。

这个“排队”用两层机制保证：

1. **第一层：应用层的锁（Redis分布式锁）**。当A的“加入”请求进来时，先尝试在Redis里给这个订单“上一把锁”（本质是往Redis里写一条带过期时间的记录，如果这条记录已经存在说明已经被别人锁住了）。如果拿到锁，才能继续往下执行；如果没拿到（说明另一个请求正在处理这个订单），就返回“当前拥挤，请重试”或者等待。这一层锁反应快，能在请求真正碰到数据库之前就把大部分并发请求挡住。
2. **第二层：数据库层的锁（MySQL行锁）**。即使应用层的Redis锁因为某些极端情况失效了（比如系统部署了好几台服务器，Redis网络出问题），在真正读取和修改这个订单在数据库里的记录时，会用 `SELECT...FOR UPDATE` 这种查询方式，它的效果是“我在读这条记录的同时，把这条记录锁住，别的事务想读或改这条记录必须等我操作完提交”。这是最后一道防线，即使前面的锁失效了，数据库层面依然能保证同一时刻只有一个操作真正在改这条记录。

两层锁配合的意义：Redis锁负责“快速挡掉大部分并发，减轻数据库压力”，MySQL行锁负责“即使前面失效了，也绝对不会出现数据错乱”。

加入成功后的原子三步

拿到锁之后，系统按顺序做三件事，这三件事被绑定在同一个数据库事务里（意味着这三步要么全部成功、要么全部失败回滚，不会出现“扣了钱但没加上名额”这种中间状态）：

1. 重新查一次当前订单的剩余名额（因为可能在等锁的过程中，状态已经变了）
2. 确认还有名额之后，把这个用户要付的钱从他的钱包余额里冻结（不是直接转走，是先标记“冻结”，等真正核销交接完成才会真正转给团长）
3. 把订单的“当前已加入人数”加1

三步做完提交事务后释放锁，让排在后面的请求继续处理。

幂等：防止用户手抖点了两次

如果用户网络卡顿，同一个"加入"请求被前端重复发送了两次，系统不能真的执行两次扣款。做法是给每一次"加入"操作生成一个唯一标识，这个标识作为数据库表里的一个**唯一索引**字段。第二次重复请求进来时，尝试插入这条记录会因为唯一索引冲突而失败，系统捕获到这个"冲突"，就知道"这个操作之前已经执行过了"，直接把第一次执行的结果返回给用户，不会真的再扣一次钱。

流程四：核销交接与退款——阶梯规则怎么落地、钱怎么准确地转

整体链路

订单进入不同阶段 → 【查询当前阶段允许的退款规则】 → 【执行退款/扣违约金/禁止退出】 → 【信用分变动】
核销时 → 【验证核销码】 → 【冻结金额转入团长账户】 → 【扣服务费】

阶段规则怎么实现

订单一生有四个主要阶段：集资中、购买中、待交接、已完成。每个阶段对"能不能退出、退多少钱"的规则是不一样的（集资中随便退全额退、购买中退要扣违约金、待交接不能退）。

实现方式是把这些规则**写死在"订单状态"这个概念本身自带的判断逻辑里**，而不是散落在各个功能的代码里各自判断一遍"现在是什么状态、能退多少"。这样做的好处是：任何地方需要知道"这个订单现在能不能退款、该退多少"，都是问同一处逻辑要答案，不会出现"退款功能这里判断的规则"和"订单详情页显示的规则"不一致的情况。

逆向流转：一个容易被忽略的边界情况

假设一个拼单原本需要5人，已经满员进入"购买中"阶段，这时有1个人申请退出（购买中允许退出，但要扣违约金）。退出之后，这个订单只剩4个人了，不满足"5人才成团"的条件——这个订单现在应该怎么办？

系统的处理方式是：退款流程执行完之后，**主动检查一次"退款后剩余人数是否还满足原定人数要求"**，如果不满足，就把订单状态从"购买中"重新降级回"集资中"，同时重新对外开放"加入"入口，让新的人可以顶上这个空出来的名额。这避免了订单卡在一个"人数不够但状态又停在购买中"的尴尬状态里。

金额计算为什么用Decimal，不用float

计算机里的普通小数（float）用二进制来表示十进制小数，会有精度损失（比如0.1在二进制里其实是个无限循环小数，只能被近似存储）。如果拿float做多次加减乘除，误差会累积，涉及金额的场景哪怕一分钱的偏差都不能接受。所以所有金额相关的计算，都统一用一种叫Decimal（精确小数）的数据类型，它内部按十进制精确存储数字，不会有这种精度丢失问题。算出服务费之类的结果后，还会强制"保留两位小数"（比如算出3.567，会被强制处理成3.57或3.56，按四舍五入或者截断规则统一处理，避免出现三位小数的钱）。

余额变更为什么不是"先读、再改、再写回去"

如果余额更新的逻辑是"先把当前余额读出来 → 在程序里加上或减去这次变动的金额 → 把新余额写回数据库"，这中间存在一个风险：如果两个操作几乎同时对同一个账户做余额变更（比如同时有两笔钱要转进这个账户），两边都基于"读出来的那个旧余额"去计算新余额，最后谁写回去晚，就会覆盖掉另一笔操作的结果，导致其中一笔钱"消失"。

系统的做法是**让数据库自己去做"加减"这个动作**，写成类似"把余额字段直接加上这个金额，前提是当前余额不能变成负数"这样一条数据库更新语句，而不是程序先读再算再写。数据库在执行这条更新语句的时候，本身就保证了这个"读取当前值+计算+写回"的过程是不可分割的一个原子操作，不会被其他并发的更新语句插进来打断。

信用分机制

退出拼单会扣信用分，且**阶段越晚扣得越多**（集资阶段退出扣1分，购买阶段退出扣3分）——这是用经济和信誉成本去抑制"随便报名又随便退出"的行为，越接近实际购买/交付，退出对其他拼友造成的影响越大，代价也应该越高。

流程五：现货转让小票识别 —— 拍照怎么变成商品清单

整体链路

用户拍摄/上传小票照片 → 【图片编码上传】 → 【AI多模态识别】 → 【结果解析成商品列表】 → 【生成分摊清单】

阶段①：图片怎么传给AI

用户拍的小票照片本质是一份图片文件，要把它交给能"看图"的AI模型去识别，需要先把图片转换成一种能在网络上传输的文本格式（Base64编码，即把图片的二进制数据转成一长串字符），然后上传到Dify这个平台提供的一个"文件上传"接口，Dify会返回一个文件的唯一标识，后续调用识别任务时用这个标识去引用这张图片，而不需要重复传整个文件内容。

阶段②：为什么用Dify这个中间层，而不是直接调大模型接口

小票识别本质是"给一张图，模型要看懂图里的文字和排版结构，抽取出里面每一行商品的名称、单价、数量、总价"，这是一个"多模态"任务（同时处理图像和文字）。Dify是一个可以用**可视化方式搭建AI处理流程**的平台，你可以在它的界面上配置："输入是一张图片，先用什么模型识别，识别完再用什么规则整理输出格式"，这整套流程被称为一个"工作流"。业务代码不需要关心这个工作流内部具体调用了哪个模型、用了什么提示词，只需要把图片传进去、等结果出来。这样做的好处是：如果以后想换一个识别效果更好的模型，或者想调整提示词让识别更准，只需要在Dify的界面上改配置，不需要改动和重新部署业务代码。

阶段③：识别结果的整理（这是实际做起来最麻烦的一步）

Dify的工作流最终会返回一份数据，理论上应该是一份结构清晰的"商品数组"，但实际调用下来发现，返回的数据格式并不总是稳定：有时候直接是一个数组；有时候数组被包在一个字典里，字典只有一个键，值才是真正的数组；有时候这个数组本身又被转成了一整段文本字符串（本该是结构化数据，结果变成了"看起来像JSON的一段字符串"）。

处理方式是写一层"层层剥壳"的判断逻辑：先看拿到的数据是什么类型，如果是字符串就尝试当成JSON去解析，解析出来再看是不是数组，如果外面包了一层字典就把字典的值取出来再判断一次……逐层剥到最后拿到真正的商品列表。而且这个过程中如果某一条商品的数据格式实在有问题解析不出来，系统只会跳过这一条并记录日志，不会因为一条数据有问题就让整个识别任务判定失败——因为一张小票可能识别出十几个商品，不能因为其中一条格式有点问题就让用户重新拍照，把能用的结果先给用户是更好的体验。

流程六：纠纷处理 —— AI怎么判责、钱怎么自动结算

（这部分已经在之前解释过异步部分，这里完整梳理一遍全流程）

整体链路

用户提交纠纷（描述+证据图） → 【落库+发消息】 → 【后台异步处理】 → 【AI判责】 → 【合规校验】 → 【按置信度决定自动结算或转人工】

阶段①：提交纠纷，先记录再处理

用户在订单交接后发现问题（比如货不对版），填写纠纷描述并上传1-3张证据照片提交。这个提交请求要做的第一件事很简单：把这条纠纷信息存进数据库，状态标记为"处理中"（PENDING）。

在存库的同时，系统会往消息队列（RabbitMQ）里发一条消息，内容大概是"有个新纠纷，ID是多少，需要去处理"。发完这条消息，这个提交请求就可以立刻给用户返回"提交成功"了——**注意此时AI判责这件事还完全没有开始做**，用户不需要等AI处理完才能看到"提交成功"的反馈。

阶段②：后台独立进程接手（消费者/consumer）

系统里有一个专门处理"纠纷相关AI任务"的后台常驻程序，它不接收用户的网页请求，而是一直"盯着"消息队列，一旦发现队列里有新消息，就把它取出来处理。它拿到"纠纷ID"后，会去数据库把这条纠纷的详细信息（描述文字+证据图片）都查出来，准备调用AI。

为什么要绕这一圈而不是在提交接口里直接调AI：AI处理（尤其是多模态图片理解）往往需要几秒甚至更久，如果让用户提交纠纷这个网页请求一直等着AI跑完才返回，用户体验很差，网络请求也容易超时。把这个"慢的部分"挪到后台独立进行，用户提交完就可以先去做别的事，前端可以之后再来查询处理结果。另外，如果这次AI调用刚好失败了（比如AI服务临时抽风），因为消息还在（或者可以重新放回）队列里，可以选择重试，不会因为一次失败就彻底丢了这个处理任务。

阶段③：AI判责（多模态推理）

后台程序把纠纷描述文字和证据图片一起传给Dify配置的“纠纷裁判” workflow。这个 workflow 背后调用的是一个支持“看图+读文字”的多模态大模型，模型会综合描述内容和图片证据，给出一个判断：这次纠纷责任应该归团长、归拼友、还是双方各承担一部分，并且给一个“我对这次判断有多大把握”的置信度分数。

阶段④：拿到AI结果后，不是直接执行，要先过两道关

第一道关：涉及金额是否超过阈值。在AI真正被调用之前（准确说是在提交纠纷的一开始），系统就会先检查这个订单涉及的金额，如果达到或超过500元这个阈值，系统直接判定“这个纠纷金额太大，不适合让AI自动裁决”，跳过整个AI判责环节，直接转成人工审核状态。这一步的逻辑是：AI判责终究有出错的可能，涉及的金额越大，一旦判错造成的实际损失就越大，所以给AI设定一个“能自主决定的金额上限”，超过这个上限的一律走人工。

第二道关：AI输出的结果本身是否“合理”。AI返回的判责结果里，会包含团长应承担的责任比例和拼友应承担的责任比例这两个数字，逻辑上这两个数字加起来应该正好等于1（比如团长70%责任、拼友30%责任）。系统会做一次校验，检查这两个数字加起来是否等于1（允许一个很小的误差范围，因为数字在传输和解析过程中可能有极其微小的偏差）。如果加起来的结果明显不等于1（比如加起来是1.3或者0.6），说明AI这次的输出本身存在逻辑错误，系统会直接拒绝采纳这个结果，不会拿一个数学上就说不通的比例去划钱。

阶段⑤：根据置信度决定“自动结算”还是“转人工”

过了上面两道关之后，还有最后一道关卡——**置信度**。AI在给出判责结果的同时，也会给一个“这次判断我有多大把握”的分数。系统设定了一个阈值（80%）：

- **置信度达到80%以上：**系统认为这个判断足够可信，直接按判责结果自动执行资金操作——如果是团长全责，就把冻结的钱从团长的账户退还给拼友；如果是拼友全责，就把违约金从冻结金额里划给团长；如果是双方各担一部分责任，就按各自的责任比例分别计算金额并划转。
- **置信度低于80%：**系统不会自动执行任何资金操作，只会把AI给出的判责建议（判责方、比例、置信度、AI给出的判断理由）存进这条纠纷记录里，供人工客服后续处理时参考，但最终这笔钱怎么划，由人工来决定。

阶段⑥：资金划转的幂等保护

不管是自动结算还是人工后续操作，每一次实际的资金划转都会带一个独立的、唯一的标识（幂等键）。这个设计是为了防止“消息队列可能会把同一条处理任务重复投递”这种情况——万一因为网络或系统原因，同一个纠纷的结算任务被消费者处理了两次，靠这个唯一标识，第二次执行时系统能识别出“这笔钱之前已经划过了”，不会重复划一次。

流程七：账户与资金安全 —— 敏感信息怎么保护、登录怎么防盗用

场景一：手机号/收款账户这类敏感字段怎么存

问题：这些字段如果直接以原文形式存在数据库里，一旦数据库被攻破（比如被拖库），所有用户的手机号、收款账户就直接泄露了。

实现方式：用一种叫AES-256-CBC的对称加密算法，把这些字段的原始内容加密后再存进数据库。"对称加密"的意思是加密和解密用的是同一个密钥，只有拥有这个密钥的系统才能把加密后的乱码还原成原始内容，数据库里存的是加密后的乱码，即使泄露出去，没有密钥也没法读出真实的手机号。

加密过程还加了两个细节处理：**每次加密都会用一个随机生成的"初始化向量"**（这样即使是同一个手机号，两次加密出来的结果也不一样，避免被人通过"密文相同就代表原文相同"这种规律反推出信息）；对不满足加密算法要求的固定长度的原文做**填充处理**（PKCS7填充），凑够加密算法要求的数据块大小。

衍生出的一个问题：加密后怎么查找。比如登录时要用手机号去数据库里查这个用户，但数据库里存的是加密后的乱码，同一个手机号两次加密结果还不一样，没法直接用"手机号=某个值"去匹配查询。解决方式是**额外维护一个哈希字段**：把手机号用SHA256这种"单向不可逆"的哈希算法处理一遍，得到一个固定的、每次结果都一样的哈希值，存进数据库另一个字段专门用来做查找匹配。查找时把输入的手机号同样做一次哈希，去数据库里匹配这个哈希字段，匹配上了再去用加密字段还原出真实手机号做进一步业务处理。这样既能保证敏感信息本身是加密存储的，又不影响正常的查找功能。

场景二：密码怎么存

密码和手机号的需求本质不同：手机号业务上有时需要"读出原文"（比如要给用户发短信），密码永远不需要被还原出原文，只需要验证"用户输入的密码和当年注册时设的密码是不是一致"。

因为不需要还原原文，密码用的是**哈希算法（bcrypt）**而不是加密算法。bcrypt的特点是：同一个密码每次生成的哈希值都不一样（内部自带随机"盐值"），而且故意设计得"计算很慢"（这是有意为之——如果哈希算法算得太快，攻击者拿到泄露的哈希值后可以疯狂穷举尝试所有可能密码去撞库，算得慢会大幅拖慢这种暴力破解的速度）。验证密码时，不是把存的哈希值解密还原成密码去比较，而是把用户刚输入的密码重新做一次同样的哈希运算，比较两次的哈希结果是否一致。

如果运行环境不支持bcrypt（某些特殊部署环境可能缺少依赖），系统会降级用HMAC-SHA256这种哈希方式顶上，但这个降级方案专门用了另一套独立的密钥，不会和前面存手机号用的AES密钥共用——这样即使某一个密钥不小心泄露了，也只会影响它对应的那一套机制，不会一泄露就波及两边。

场景三：登录状态怎么维持，怎么实现"新设备登录挤掉旧设备"

基本机制：用户登录成功后，系统发给他两个凭证（Token）：一个短期有效的"访问令牌"（Access Token，比如7天有效），日常操作都带着这个令牌证明身份；一个长期有效的"刷新令牌"（Refresh Token，比如30天有效），专门用来在访问令牌快过期或已过期时换取一个新的访问令牌，不需要用户重新输入密码登录。

新旧设备互踢怎么实现：每个令牌里都编码了一个"版本号"。系统在Redis里给每个用户维护一个"当前有效版本号"。每次用户重新登录（比如换了新手机登录），Redis里这个版本号就自增一次，同时新发出去的令牌里带着这个新的版本号。而旧设备上还留着的、版本号是旧的那个令牌，下次用它发请求时，系统会拿令牌里的版本号跟Redis里当前的版本号做比较，发现不一致，就判定这个令牌已经失效，旧设备就此被登出——这就是"最新登录的"

设备生效，之前的设备自动掉线"的实现方式。

一个安全取舍：如果Redis服务本身出故障、连不上了，理论上系统可以选择"干脆先放行别管版本号校验了"（保证功能能用），也可以选择"连不上就直接拒绝，不允许登录态生效"（保证安全但牺牲了可用性）。这里选择的是后者——**安全优先于可用性**，宁可这段时间大家都用不了，也不冒险在校验机制失灵的情况下还放行请求。

场景四：接口限流防暴力破解

登录接口按"来源IP"做频率限制，比如同一个IP地址60秒内最多允许尝试10次登录。这个限制通常也是用Redis实现的：每次有登录请求，就以这个IP为标识在Redis里计一次数，并设置60秒的过期时间，超过10次就直接拒绝这个IP接下来的请求，逼迫攻击者放慢速度，大幅提高暴力破解密码所需要花费的时间成本。